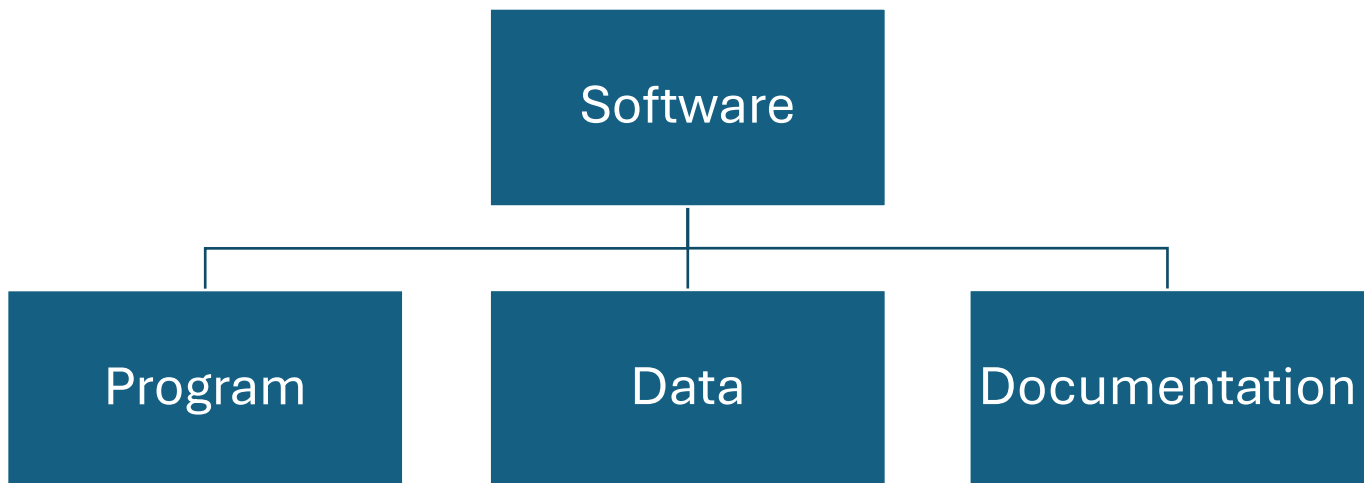


➔ **Software:** Computer Instructions or data



It is not just code. It includes:

1. **Programs** → Executable instructions.
2. **Documentation** → Manuals, user guides, requirements docs.
3. **Data** → Input/output handled by software.

Example: A banking software → code (transactions), documentation (how to use), data (accounts, balance).

➔ **Engineering:** The Process of productive use of scientific knowledge is called Engineering.

➔ **Software Engineering:** Application of systematic, disciplined and quantifiable approach to development, operation and maintenance of software.

**** Why do we need Software Engineering?**

We need software engineering because only coding is not enough, we need to build software that are reliable, scalable, secure, maintainable and solve real world problems.

Here are the key reasons:

- **Managing Complexity**
- **Cost & Time Efficiency**
- **Reliable and High Quality**
- **Scalability and Maintainability**
- **Team Collaboration**
- **Security & Safety**
- **Meeting User Needs**

**** Difference Between CS and SE:**

➔ **CS:** Focuses on theory + algorithms, programming languages, data structures, AI, operating systems, databases, networking, machine learning, cybersecurity, etc.

Example: A CS person might invent a new sorting algorithm or optimize existing ones

➔ **SE:** Focuses on practical and systematic development of software systems, deal with software processes, project management, SDLC models, requirement analysis, testing, maintenance, etc.

Example: An SE person might use an existing sorting algorithm as part of Hospital management system.

Aspect	Computer Science (CS)	Software Engineering (SE)
Focus	Theory & fundamentals of computing	Application of engineering to software
Nature	Research & innovation	Practical implementation
Approach	Algorithms, computation, AI	Development lifecycle, project management
Output	Knowledge, models, algorithms	Working, reliable software systems
Career Paths	Researcher, Data Scientist, AI Engineer	Software Developer, Architect, Project Manager

**** Problems in Software Development:**

- ⇒ The final software does not fulfill the needs of the customer
- ⇒ Hard to extend and improve: if you want to add a functionality later its mission impossible
- ⇒ Bad documentation
- ⇒ Bad quality: frequent errors, hard to use, ...
- ⇒ More time and costs than expected

****Importance Of Software:**

Society is becoming increasingly dependent on software components e.g.

- ⇒ *electronic devices* (microwave oven, washing machine etc.)
- ⇒ *vehicles* (cars, buses etc.)
- ⇒ *mission critical systems* (missiles, aircraft etc.)
- ⇒ *medical machines* increased complexity because of embedded software.

**** Software Bugs:**

A **software bug** is an error, flaw, or defect in a program that causes it to behave unexpectedly, produce incorrect results, or crash.

➔ History Of Software Bugs:

1. NASA Mariner 1 (1962)

- A missing **hyphen in the code** caused the spacecraft to veer off course.
- Result: Spacecraft destroyed → \$18 million loss.

2. Ariane 5 Rocket Failure (1996)

- Software reused from Ariane 4 without proper testing.
- A **data conversion error** led to rocket explosion after 37 seconds → \$370 million loss.

3. Therac-25 Radiation Machine (1985–87)

- Software bug caused patients to receive **massive overdoses of radiation**.
- Result: Several deaths and serious injuries.

4. Y2K Bug (Year 2000 Problem)

- Dates stored with only 2 digits (e.g., “99” instead of “1999”).
- Fear: Systems would crash in 2000 thinking it’s 1900.
- Result: Billions spent on fixing code worldwide.

5. Pentium Bug:

- Occurred in Intel’s Pentium processors due to an error in the floating-point division unit.
- Result: The processor gave incorrect decimal results for certain division operations.
- Impact: Loss of trust in Intel, replacement of faulty chips → \$475 million cost.
- Example: Dividing $4194835 \div 3145727$ gave 1.333474 instead of 1.33382

6. Friendly Fire:

- Happened in a Patriot Missile Defense System during the Gulf War.
- Bug in **time calculation software**: the internal clock drifted due to rounding errors.
- Result: System failed to track an incoming Iraqi Scud Missile
- The Scud hit the U.S Army Barracks in Dhahran, Saudi Arabia → 28 soldiers killed.

**Well-Engineered Software:

- Reliable
- Good User Interface
- Acceptable Performance
- Good Quality
- Cost-Effective
- Scalable

**Goals of Software Engineering:

- Meeting users' needs
- Low cost of production
- High performance
- Portability
- Low cost of maintenance
- High reliability
- Deliver on time

**What does a Software Engineer do?

Software Engineer apply the concepts and techniques of computer science, engineering, mathematical analysis to the design, development testing and maintenance of software and systems.

**Types of Software Systems (Software Engineering Diversity):

1. Standalone Applications:

- Run on a single computer/device without needing a network connection.
- Installed locally (desktop software, mobile apps).
- Examples: Microsoft Word, Photoshop, VLC Media Player.

2. Interactive Applications:

- Applications where users perform **transactions** (queries, updates, purchases) through a user interface.
- Usually client–server or web-based.
- Examples: Online banking systems, e-commerce websites (Amazon, Daraz).

3. Embedded Applications:

- Software is built into **hardware devices** to control their functions.
- Works in real-time, often safety critical.
- Examples: Software in washing machines, pacemakers, car braking systems.

4. Batch Processing System:

- Process **large volumes of data** at once (in batches), without user interaction during execution.
- Efficient for repetitive, heavy tasks.
- Examples: Payroll systems, bank cheque processing, monthly billing systems.

5. Entertainment Systems:

- Designed for fun, leisure, and media consumption.
- Can be software-only or combined with hardware.
- Examples: Video games, Netflix, Spotify, VR games

6. Systems for Modeling and Simulations:

- Software used for **scientific, engineering, or educational simulations**.
- Helps in testing, training, or predicting real-world behavior without risks.
- Examples: Flight simulators, climate change models, traffic simulations.

**Types of Software Products:

1. Generic Product:

- These are **ready-made, mass-market software** developed for a large number of customers.
- Produced by a company and sold to anyone who wants them.
- Features are decided by the software company, **not by individual users**.

Key Points:

1. Developed once → sold to many customers.
2. Usually available off-the-shelf or downloadable.
3. Users can configure settings, but **core features are fixed**.
4. Examples: Microsoft Office, Adobe Photoshop, Antivirus software, WhatsApp.
5. Target → **wide audience** with common needs.

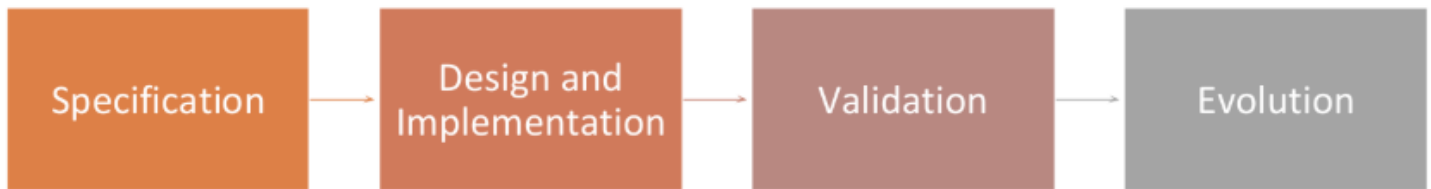
2. Customized Product:

- These are **tailor-made software solutions** developed for a specific organization or customer.
- Built after understanding **unique requirements**.
- Features are decided **entirely by the client's needs**.

Key Points:

1. Developed for **one customer or organization** only.
2. More costly & time-consuming than generic software.
3. Offers exactly what the customer needs (flexibility).
4. Examples:
 - Online Banking System for a specific bank
 - University's Student Management System
 - Hospital Billing System
5. Target → **specific organization/individual**.

**Software Process:



1. Specification (Requirements Engineering)

👉 What should the system do.

- Gather requirements from client/users.
- Define system's functionality, features, and constraints.
- Create **SRS (Software Requirement Specification)**.
 - ✅ *Example:* In an online banking app, specify "Users can transfer money, check balance, pay bills."

2. Design and Implementation

👉 How the system will be built.

- **Design:** Create architecture (overall structure), database design, UI design.
- **Implementation (Coding):** Convert design into actual source code.
 - ✅ *Example:* Choosing a 3-tier architecture for a banking system, designing tables for users, then coding in Java/Python.

3. Validation (Verification & Validation)

👉 Making sure the system does what it should.

- **Verification:** Are we building the product right? (check against design/specs)
- **Validation:** Are we building the right product? (check against customer needs)
- Includes testing (unit testing, integration testing, system testing, acceptance testing).
 - ✅ *Example:* Testing a money transfer feature to ensure correct balance updates.

4. Evolution (Maintenance)

👉 Software always changes after delivery.

- Fixing bugs found later.
- Adding new features as business grows.
- Adapting software to new environments (e.g., new OS, devices).
✅ *Example:* Adding QR-based payments in the banking app after launch.

**SDLC (Software Development Life Cycle):

- **SDLC** stands for **Software Development Life Cycle**.
- It is a **step-by-step process** followed by software organizations to **design, develop, test, and maintain software systems**.
- The main goal of SDLC is to **deliver high-quality software** that meets customer requirements within time and budget.
- Think of it as a **roadmap/blueprint** for building software in a structured way.

Phases of SDLC:

1. Requirement Analysis

- Gather and study what the client or user needs.
- Both **functional requirements** (what the system should do) and **non-functional requirements** (performance, security, usability).
- Example: For an online shopping site, requirements could be: "Add to cart, secure checkout, payment gateway."

2. System Design

- Create a **blueprint** of the system.
- Includes **high-level design (HLD)** → overall architecture, and **low-level design (LLD)** → database schema, algorithms, interface details.
- Example: For online shopping, design the login page, product catalog, database for users & orders.

3. Implementation (Coding)

- The actual development of software starts.
- Developers write code in the chosen programming language (Java, Python, C++, etc.).
- Example: Writing the backend API for product search and frontend code for displaying products.

4. Testing (Verification & Validation):

- The system is tested to check if it meets requirements and is **bug-free**.
- Types of testing: Unit testing, Integration testing, System testing, User Acceptance Testing (UAT).
- Example: Check if the shopping cart updates correctly when items are added or removed.

5. Deployment:

- The software is released to the customer or the market.
- Can be done in phases (beta release, pilot testing) or full release.
- Example: Launching the shopping website live on a hosting server.

6. Maintenance (Evolution):

- After release, software needs updates, bug fixes, and improvements.
- Example: Adding new payment methods, fixing security issues, or improving performance.